

Modification and Improvement of a Coffee Berry Sorting Machine

Continuous Assessment Presentation — FYP 22-22

Joe Annel Prosper Muuo

March 2026

Dept. of Mechatronic Engineering, JKUAT

Supervisor: Dr. Maureen Andanje

Main Objective

To modify and improve an existing automated coffee berry sorting machine.

Specific Objectives:

1. To analyse the performance limitations of the existing image-based coffee berry sorting system, with emphasis on motion blur, processing latency, and classification accuracy.
2. To improve on the sorting mechanism.
3. To evaluate alternative or upgraded processing hardware capable of supporting higher frame rates and faster inference.

Three-area improvement strategy:

1. Performance audit of the original system
2. Targeted hardware and mechanism upgrades
3. Upgraded processing platform with retrained model

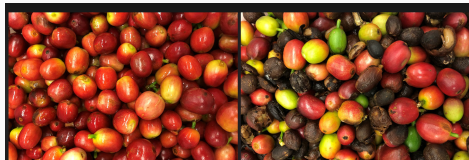
Key design decisions:

- Binary accept/reject classification — eliminates semi-ripe/ripe boundary confusion
- Global-shutter camera — eliminates rolling-shutter motion blur

Bi-4 + FSD32 as processing offload



Visual diversity demanding robust classification



Methodology — Dataset & YOLOv11 Model Training

Dataset:

- 2,000 images annotated on Roboflow
- Split: 70% train / 15% val / 15% test
- Sources: controlled LED tabletop + in-situ chute frames
- Camera: Arducam OV9281 (global shutter, 1 MP)
- Resized to 640×640 for training

YOLOv11n Training Config:

- Backbone: yolo11n.pt (COCO pre-trained, fine-tuned)
- 100 epochs, batch 16, SGD $\text{lr} = 0.01$



YOLOv11 validation batch predictions

Methodology — Hardware Upgrades

Three targeted mechanism upgrades:

1. **Camera:** Pi Camera V2 → Arducam OV9281 (global shutter)

Eliminates motion blur at 0.5 m/s chute speed

2. **Ejector driver:** PN2222A BJT → IRLZ44N MOSFET

No storage time; full drive from 3.3 V ESP32 GPIO

3. **Co-processor:** ESP32 added

Handles solenoid timing, motor PWM, relay switching
with $<5\ \mu\text{s}$ ISR latency

Components procured & bench-verified:

- Raspberry Pi 4 (4 GB RAM)
- Arducam OV9281 global-shutter module
- ESP32 DevKit
- LED ring light
- 2-channel optocoupler relay (PC817)
- IRLZ44N MOSFETs $\times 4$, 1N5819 diodes

Co-processing:

Pi 4 $\xrightarrow{\text{UART } 115200}$ ESP32 → solenoids / PWM / relay

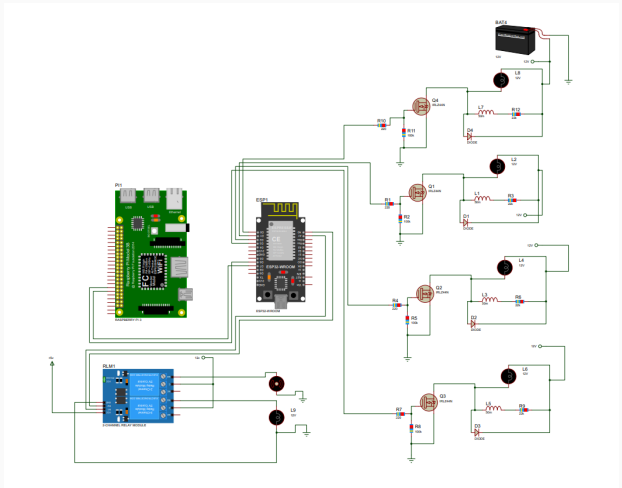
Methodology — Circuit Verification in Proteus

Simulation scope:

- 4× IRLZ44N solenoid driver channels
- Motor PWM stage (GPIO 14)
- Compressor relay (GPIO 21)
- LED strip driver (GPIO 22)
- UART virtual terminal (Pi link at 115 200 bps, 8N1)

Before any hardware built:

ESP32 firmware loaded into virtual ESP32-WROOM-32; classification commands



Complete Proteus ISIS schematic: ESP32, 4× MOSFET ejector channels, relay, and UART terminal

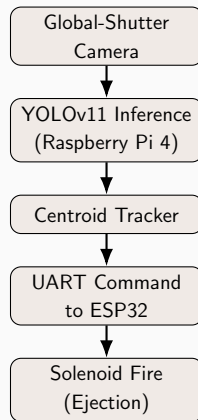
Methodology — Detection-to-Actuation Pipeline

End-to-end pipeline flow:

1. Global-shutter frame capture (detection zone)
2. YOLOv11 inference on Raspberry Pi 4
3. Centroid tracker associates detections across frames
4. Berry reaches ejection zone → Pi sends ACCEPT or REJECT: n via UART
5. ESP32 fires solenoid jet for calibrated pulse duration

Target latency: <100 ms frame-capture to solenoid trigger

Baseline (existing): ≈ 380 ms



Results — Baseline Performance Audit

Existing system measured:

- End-to-end latency: ≈ 380 ms
(target < 100 ms)
- Classification accuracy: $\approx 72\%$
(target $> 90\%$)
- Throughput: ≈ 35 berries/min
(target > 150)

Root causes identified:

- 310 ms serialised TFLite inference on Pi 3B+
- Rolling-shutter smear degrading colour classification at chute speed
- Four class model — semi-ripe/ripe

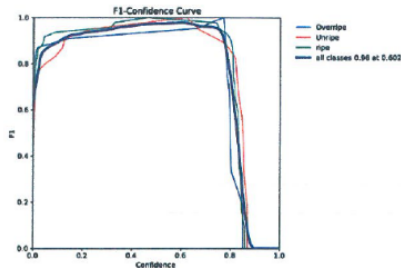
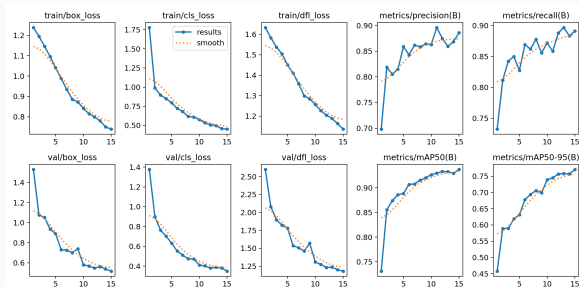


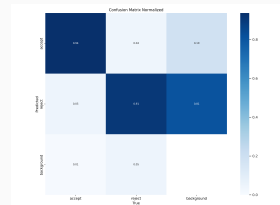
Figure 4.13: F1 score

Previous 4-class model F1 score —
misclassification at semi-ripe/ripe boundary

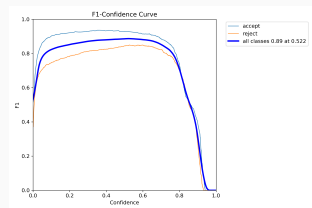
Results — YOLOv11n Training & Validation



Training curves: loss, precision, recall, mAP@0.5
over 100 epochs



Normalised confusion matrix

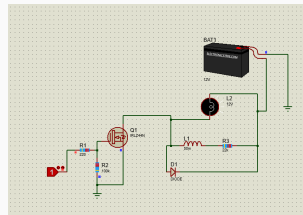


F1-confidence curve

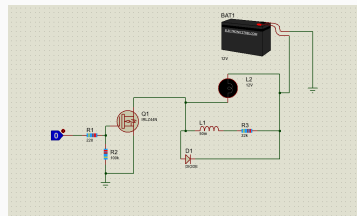
Results — Proteus Circuit Simulation

Key verified results:

- All 4 MOSFET channels fired correctly on REJECT:n commands
- UART parsed at 115 200 bps within 10 ms simulated latency
- No gate ringing — 220 Ω gate resistor effective
- Flyback diode clamped spike to +12.5 V (0.5 V above rail)
- 100 k Ω pull-down: zero false firing at power-on reset
- Motor PWM 0–100% confirmed proportional voltage control



MOSFET turn-ON: gate 3.3 V, drain falls to ≈ 0 V



MOSFET turn-OFF: flyback diode clamps

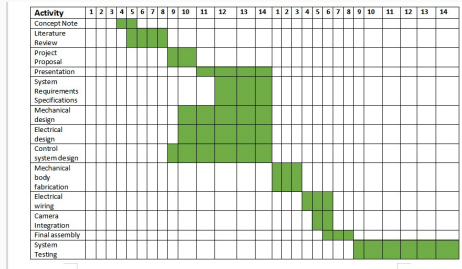
Pending Work

Model & Software

- Export YOLOv11n to ONNX/TFLite; benchmark on Pi 4
- Quantise if needed to achieve <100 ms per frame
- Implement centroid tracker for cross-frame detection
- Finalise Pi ↔ ESP32 UART protocol

Hardware & Integration

- Mount OV9281 camera above conveyor detection window
- Calibrate belt-speed-to-ejection-timing offset
- Wire solenoid drivers; full system integration



Project Gantt chart

Thank you

Questions?